

主讲老师：Fox

influxdb版本：v2.7

官网文档：<https://docs.influxdata.com/influxdb/v2/>

1.InfluxDB介绍

1.1 什么是InfluxDB

InfluxDB 是一个由 InfluxData 开发的**开源时序型数据**。它由 Go 语言写成，着力于高性能地查询与存储时序型数据。InfluxDB 被广泛应用于存储系统的监控数据，IoT（物联网）行业的实时数据等场景。

时序数据：跟时间有关数据称之为时序数据

InfluxDB 有三大特性：

- Time Series（时间序列）：你可以使用与时间有关的相关函数（如最大，最小，求和等）
- Metrics（度量）：你可以实时对大量数据进行计算
- Eevents（事件）：它支持任意的事件数据

☐ include secondary database models

44 systems in ranking, May 2024

Rank			DBMS	Database Model	Score		
May 2024	Apr 2024	May 2023			May 2024	Apr 2024	May 2023
1.	1.	1.	InfluxDB +	Time Series, Multi-model i	25.83	-0.74	-4.08
2.	2.	3.	Prometheus	Time Series	8.42	+0.50	+0.99
3.	3.	2.	Kdb +	Multi-model i	7.55	-0.16	-0.48
4.	4.	5.	TimescaleDB	Time Series, Multi-model i	4.64	-0.23	-0.08
5.	5.	4.	Graphite	Time Series	4.57	-0.18	-1.68
6.	6.	7.	DolphinDB	Time Series, Multi-model i	4.13	-0.09	+0.71
7.	7.	8.	Apache Druid	Multi-model i	3.34	+0.06	+0.28
8.	8.	9.	TDengine +	Time Series, Multi-model i	2.60	-0.08	-0.35
9.	9.	12.	QuestDB +	Time Series, Multi-model i	2.52	+0.04	+0.28
10.	11.	11.	GridDB +	Time Series, Multi-model i	1.95	-0.07	-0.43

参考网站：<https://db-engines.com/en/ranking/time+series+dbms>

基本概念

以下是 InfluxDB 2.x 的一些基本概念，这些概念有助于理解和使用 InfluxDB：

1. Bucket（桶）：Bucket 是 InfluxDB 中的基本存储单元，用于组织和存储时间序列数据。它是数据的逻辑容器，用于区分和隔离不同类型或来源的数据。
2. Organization（组织）：Organization 是 InfluxDB 中的组织概念，用于隔离和管理用户、应用程序或项目。每个用户可以属于一个或多个组织，Bucket 也属于一个组织。
3. Measurement（测量值）：Measurement 是 InfluxDB 中的数据表的概念。它类似于关系型数据库中的表，用于存储时间序列数据。每个 Measurement 包含一组字段和标签。
4. Field（字段）：Field 是 Measurement 中的一种数据类型，用于存储实际的数值数据。例如，温度、湿度等是字段的例子。

5. Tag (标签) : Tag 是 Measurement 中的一种索引类型, 用于标识和过滤数据。标签通常用于存储维度信息, 例如地理位置、设备 ID 等。
6. Point (数据点) : Point 是 InfluxDB 中时间序列数据的基本单位, 它包含了时间戳、字段和标签。每个数据点都关联到一个 Measurement。
7. API 密钥和访问令牌: 为了进行安全的访问和操作, InfluxDB 2.x 引入了 API 密钥和访问令牌。API 密钥用于身份验证, 而访问令牌用于授权对数据的访问。
8. Flux 查询语言: Flux 是 InfluxDB 2.x 中的查询语言, 用于执行强大的数据处理和分析操作。它支持更灵活的查询、过滤、聚合和转换操作。
9. 仪表板和可视化: InfluxDB 2.x 提供了仪表板和可视化工具, 允许用户创建、定制和共享图表和仪表板, 以直观地展示时间序列数据。

1.2 InfluxDB应用场景

广泛应用于物联网、监控系统、日志分析、金融数据分析等。以下是一些时序数据库的常见应用场景:

1. 监控和运维: 时序数据库常用于监控系统的性能和运行状态。通过记录服务器、网络设备、应用程序等的性能指标, 管理员可以实时监测系统健康状况, 快速识别并解决问题。
2. 物联网 (IoT) : 物联网设备产生大量的时间序列数据, 包括传感器读数、设备状态和事件记录。时序数据库能够高效地存储和查询这些数据, 为物联网应用提供实时性和高吞吐量。
3. 能源管理: 时序数据库可用于跟踪能源使用情况, 记录电表读数、太阳能发电量、温度等信息。这对于能源消耗分析、可再生能源监测和节能优化非常有用。
4. 金融数据分析: 金融行业需要处理大量的交易数据、股价变动和其他市场指标。时序数据库能够提供快速的查询和分析功能, 以便实时监控市场情况和执行高频交易策略。
5. 日志分析: 服务器日志、应用程序日志等产生的日志数据通常是按时间戳记录的。时序数据库可用于存储和分析大量的日志数据, 以便追踪系统事件、调查问题和进行故障排除。
6. 工业自动化: 在制造业和工业自动化中, 时序数据库用于记录生产过程中的传感器数据、生产线状态和设备运行情况, 以实现实时监控和质量控制。
7. 环境监测: 时序数据库可用于存储气象数据、空气质量指标、水质数据等环境监测信息。这有助于研究气候变化、提高自然灾害预警系统的效能等。

1.3 为什么不用关系型数据库

- 写入性能
- 关系型数据库也是支持时间戳的, 也能够基于时间戳进行查询。但是, 从我们的使用场景出发, 需要注意数据库的写入性能。通常, 关系型数据库会采用 B+树数据结构, 在数据写入时, 有可能会触发叶裂变, 从而产生了对磁盘的随机读写, 降低写入速度。

当前市面上的时序数据库通常都是采用LSM Tree 的变种，顺序写磁盘来增强数据的写入能力。网上有不少关于性能测试的文章，同学们可以自己去看参考学习，通常时序数据库都会保证在单点每秒数十万的写入能力。

- **数据价值**

我们之前说，时序数据库一般用于指标监控场景。这个场景的数据有一个非常明显的特点就是冷热差别明显。通常，指标监控只会使用近期一段时间的数据，比如我只查询某个设备最近 10 分钟的记录，10 分钟前的数据我就不再用了。那么这 10 分钟前的数据，对我们来说就是冷数据，应该被压缩放到磁盘里去来节省空间。而热数据因为经常要用，数据库就应该让它留在内存里，等待查询。而市面上的时序数据库大都有类似的设计。

- **时间不可倒流，数据只写不改**

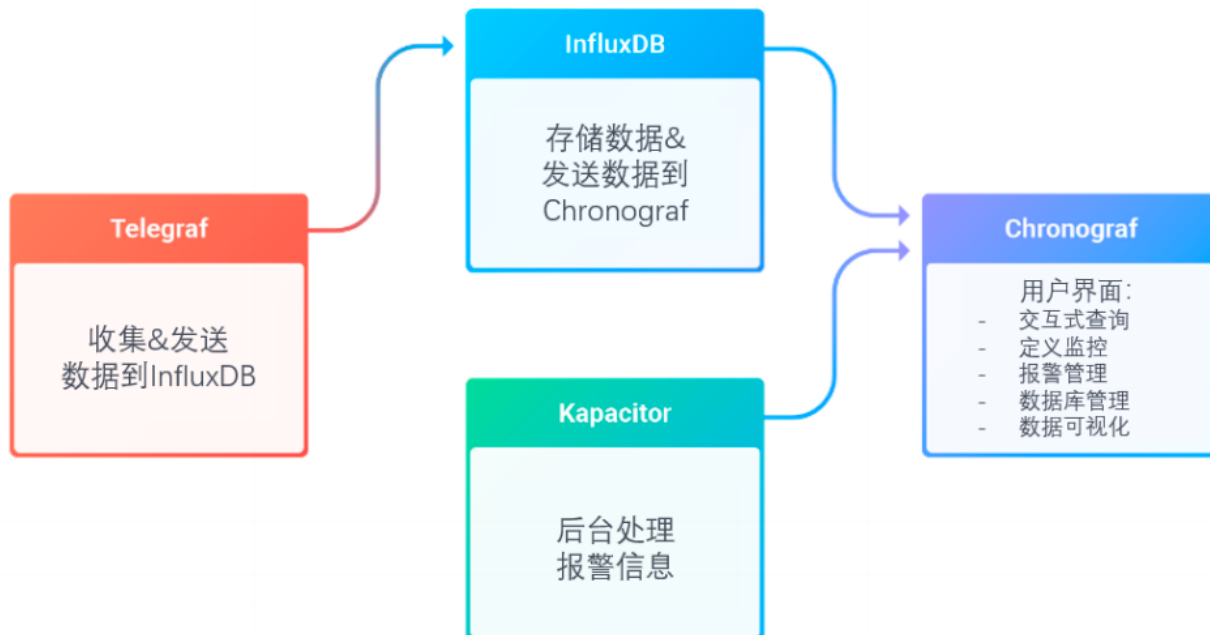
时序数据是描述一个实体在不同时间所处的不同状态。就像是我们打开任务管理器，查看 CPU 的使用情况。我发现 CPU 占用率太高了，于是杀死了一个进程，但 10 秒前的数据不会因为我关闭进程再发生改变了。这是时序数据的一大特点。与之相应，时序数据库基本上是插入操作较多，而且还没有什么更新需求。

1.4 1.X 的 TICK 技术栈与 2.X 的进一步融合

根据上文的介绍，我们首先可以知道时序数据一般用在监控场景。大体上，数据的应用可以分为 4 步走。

- 数据采集
- 存储
- 查询（包括聚合操作）
- 报警

这样一看，只给一个数据库其实只能完成数据的存储和查询功能，上游的采集和下游的报警都需要自己来实现。因此 InfluxData 在 InfluxDB1.X 的时候推出了 TICK 生态来推出start 全套的解决方案。



TICK4 个字母分别对应 4 个组件:

- T : Telegraf - 数据采集组件，收集&发送数据到 InfluxDB。
- I : InfluxDB - 存储数据&发送数据到Chronograf。
- C : Chronograf - 总的用户界面，起到总的管理功能。
- K : Kapacitor - 后台处理报警信息。

到了 2.x, TICK 进一步融合, ICK 的功能全部融入了 InfluxDB, 仅需安装 InfluxDB 就能得到一个管理页面, 而且附带了定时任务和报警功能。

1.5 时序数据库中的数据模型

想要正确使用时序数据库, 就必须理解时序数据库管理数据的逻辑。这里, 我们会和普通的 SQL (关系型) 数据库做一下比较。

传统数据库	Influxdb
数据库	bucket
表	measurement
记录	point

Point由时间戳 (time)、数据 (field)、标签 (tags) 组成。Point相当于传统数据库里的一行数据, 如下表所示:

Point属性	传统数据库中的概念
time	每个数据记录时间，是数据库中的主索引(会自动生成)
fields	各种记录值（没有索引的属性）
tags	各种有索引的属性

普通关系型数据库中的表

下面这张表示 SQL（关系型）数据库中一个简单的示例。表中有创建索引和未创建索引的列。

- park_id、planet、time 是创建了索引的列。
- _foodships 是未创建索引的列。

```
1  +-----+-----+-----+-----+
2  | park_id | planet | time | #_foodships |
3  +-----+-----+-----+-----+
4  | 1 | Earth | 1429185600000000000 | 0 |
5  | 1 | Earth | 1429185601000000000 | 3 |
6  | 1 | Earth | 1429185602000000000 | 15 |
7  | 1 | Earth | 1429185603000000000 | 15 |
8  | 2 | Saturn | 1429185600000000000 | 5 |
9  | 2 | Saturn | 1429185601000000000 | 9 |
10 | 2 | Saturn | 1429185602000000000 | 10 |
11 | 2 | Saturn | 1429185603000000000 | 14 |
12 | 3 | Jupiter | 1429185600000000000 | 20 |
13 | 3 | Jupiter | 1429185601000000000 | 21 |
14 | 3 | Jupiter | 1429185602000000000 | 21 |
15 | 3 | Jupiter | 1429185603000000000 | 20 |
16 | 4 | Saturn | 1429185600000000000 | 5 |
17 | 4 | Saturn | 1429185601000000000 | 5 |
18 | 4 | Saturn | 1429185602000000000 | 6 |
19 | 4 | Saturn | 1429185603000000000 | 5 |
20 +-----+-----+-----+-----+
21
```

InfluxDB 中的数据表示

上面的数据，如果换到 InfluxDB 中，会换一种形式进行表示。

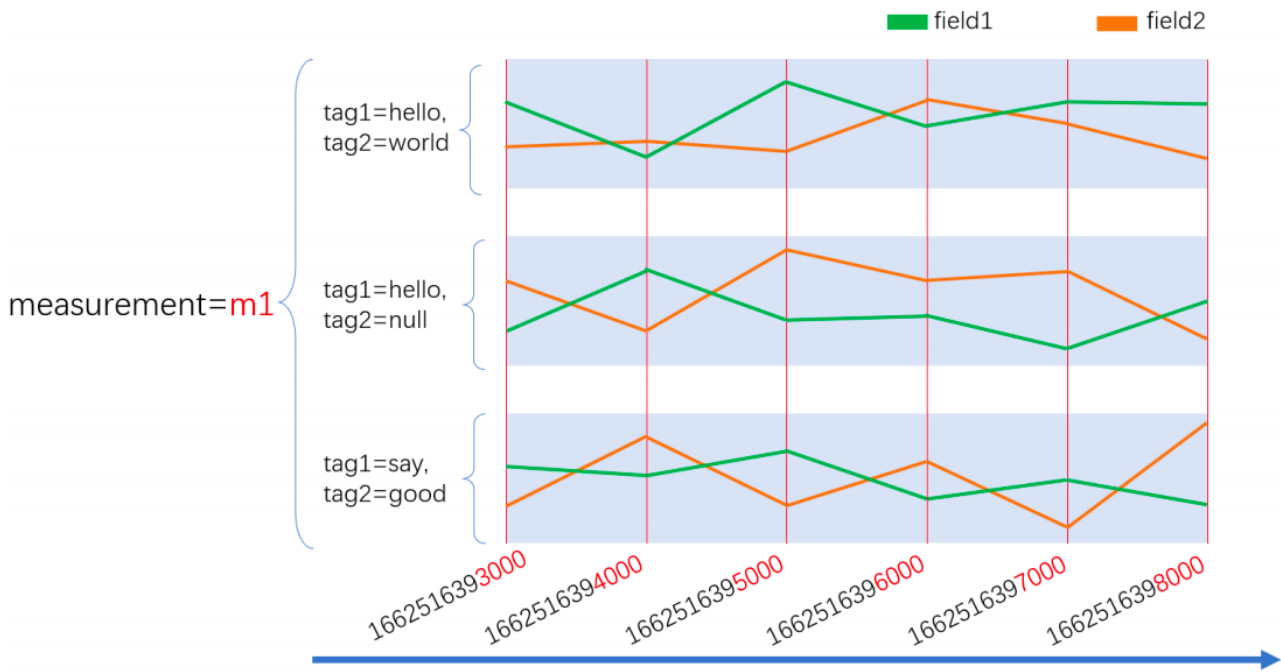
```

1 name: foodships
2 tags: park_id=1, planet=Earth
3 time #_foodships
4 ----
5 2015-04-16T12:00:00Z 0
6 2015-04-16T12:00:01Z 3
7 2015-04-16T12:00:02Z 15
8 2015-04-16T12:00:03Z 15
9 name: foodships
10 tags: park_id=2, planet=Saturn
11 time #_foodships
12 ----
13 2015-04-16T12:00:00Z 5
14 2015-04-16T12:00:01Z 9
15 2015-04-16T12:00:02Z 10
16 2015-04-16T12:00:03Z 14
17 name: foodships
18 tags: park_id=3, planet=Jupiter
19 time #_foodships
20 ----
21 2015-04-16T12:00:00Z 20
22 2015-04-16T12:00:01Z 21
23 2015-04-16T12:00:02Z 21
24 2015-04-16T12:00:03Z 20
25 name: foodships
26 tags: park_id=4, planet=Saturn
27 time #_foodships
28 ----
29 2015-04-16T12:00:00Z 5
30 2015-04-16T12:00:01Z 5
31 2015-04-16T12:00:02Z 6
32 2015-04-16T12:00:03Z 5
33

```

序列 (Series)

简单来说，InfluxDB 这类数据库是用序列的方式来管理数据的。在 InfluxDB 中，唯一的 measurement, tag_set 和 field (一个字段) 组合是一个 series (序列)。比如下图中有 6 条连续的线，这里面每个条线就是一个序列。每一个序列的数据在内存和磁盘上紧密存放，这样当你要查询这一序列的数据时，InfluxDB 可以很快定位到这一序列中的好多条数据。你也可以将 measurement, tag, field 视为索引，而且它们本身就是索引。



以序列的方式管理数据是时序数据库和传统关系型数据库最不同的地方。传统的关系型数据库通常是以 record (记录或者行) 的方式管理数据，这个时候，关系型数据库可以让你快速地通过索引定位到一条数据。

measurement	tag1	tag2	field1	field2	time
m1	hello	world	9	46	1662516393000
m1	hello	world	30	29	1662516394000
m1	hello	world	39	31	1662516395000
m1	hello	world	14	33	1662516396000
m1	hello	world	1	24	1662516397000
m1	hello	world	24	9	1662516398000
m1	hello	null	42	49	1662516393000
m1	hello	null	33	30	1662516394000
m1	hello	null	30	22	1662516395000
m1	hello	null	46	47	1662516396000
m1	hello	null	1	14	1662516397000
m1	hello	null	24	14	1662516398000
m1	say	good	28	41	1662516393000
m1	say	good	36	13	1662516394000
m1	say	good	11	2	1662516395000
m1	say	good	41	12	1662516396000
m1	say	good	28	10	1662516397000
m1	say	good	43	33	1662516398000

但是在时序场景下，我们通常需要查找某个设备最近一段时间的数据。这个时候对于传统关系型数据库来说，很可能需要多次寻址来找到多个 record 才能完成查询。而时序库是把索引打到一批次的数据上，所以在这种场景的下的读写，时序库性能是远强于 B+树数据库的。

双索引设计与高效查询思路

我们之前说到你可以将 measurement、tag_set 和 field 视为索引，还没有提到最重要的时间。其实，在 InfluxDB 中时间也是索引，数据在入库时，会按时间戳进行排序。这样，我们在进行查询时，一般遵循下面的思路。

1. 先指定要从哪个存储桶查询数据
2. 指定数据的时间范围
3. 指定 measurement、tag_set、和 field 说明我要查询哪个序列。



2.InfluxDB安装

官方文档: <https://docs.influxdata.com/influxdb/v2/install/>

2.1 linux安装

1) 下载InfluxDB二进制文件


```
1 curl -LO https://download.influxdata.com/influxdb/releases/influxdb2-  
2.7.6_linux_amd64.tar.gz  
2 tar xvzf ./influxdb2-2.7.6_linux_amd64.tar.gz  
3 sudo cp ./influxdb2-2.7.6/usr/bin/influxd /usr/local/bin/
```

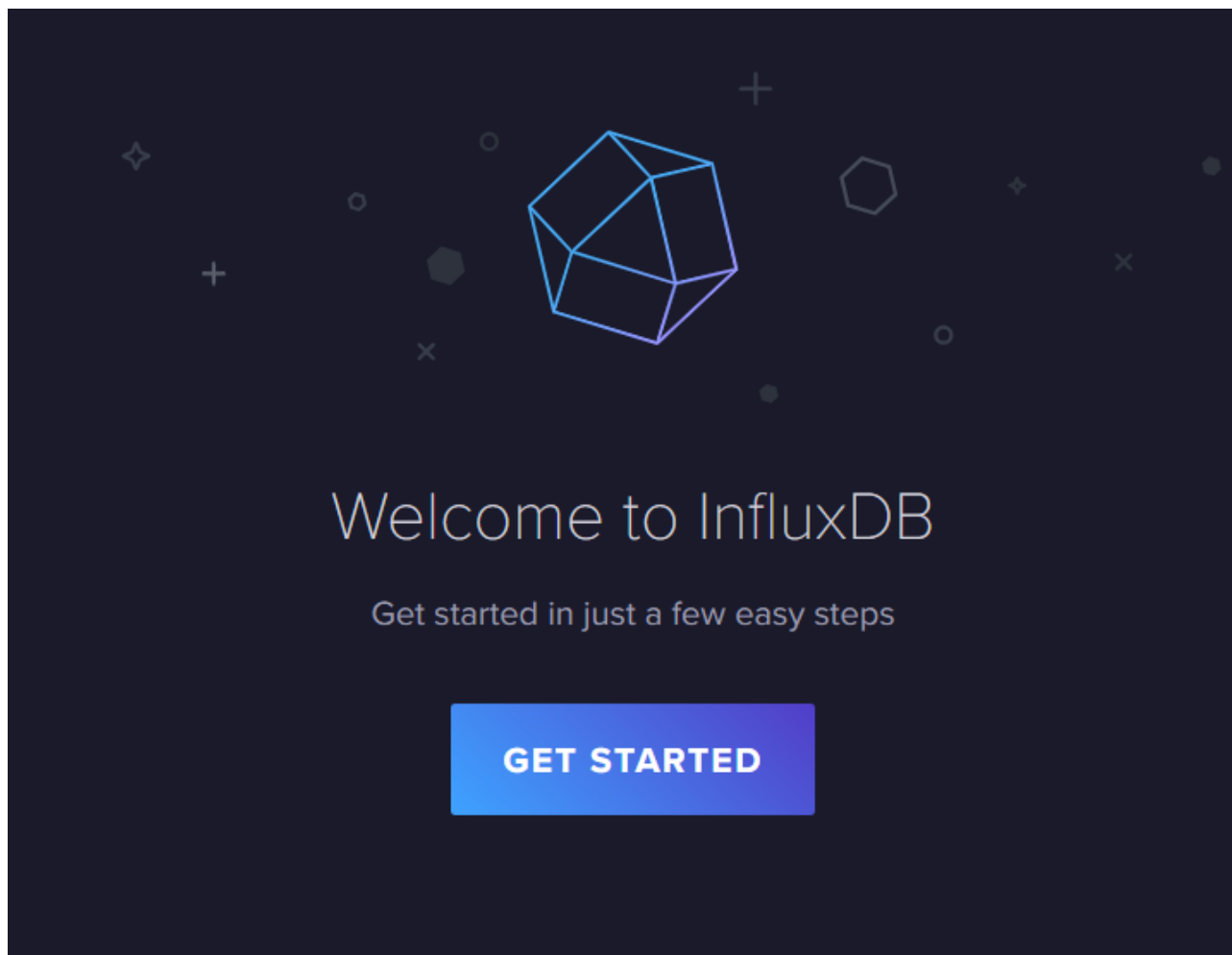
2) 启动InfluxDB

```
1 influxd  
2 #后台启动  
3 nohup influxd &
```

如果成功，可以在<http://localhost:8086>上查看InfluxDB UI

2.2 初始化配置

如果是安装后的首次使用，InfluxDB 会返回一个初始化的引导界面。按照给定步骤完成操作就好。



1) 创建用户和初始化存储桶

点击 GET STARTED 按钮，进入下一个步骤（添加用户）。如图所示，你需要填写、组织名称、用户名称、用户密码。

Setup Initial User

You will be able to create additional Users, Buckets and Organizations later

Username

fox

Password

.....

Confirm Password

.....

Initial Organization Name ?

tuling

Initial Bucket Name ?

testdb

填写完后点击CONTINUE按钮进入下一步。

2) 配置完成

You are ready to go!

Make sure to copy your operator API token now.

This token enables superuser privileges like creating users, orgs, etc. You won't be able to see it again!

NKN2TJs1IgiLd_EDouVPwPHNTRCuvbSjg9WhQ6tArR90izCk1tNMYHfkyMv_1Zh5RIpSsKaBR6e4nrT7bauw6Q==

COPY TO CLIPBOARD

Your InfluxDB has 1 organization, 1 user, and 1 bucket.

Let's start collecting data!

QUICK START

Timing is everything!

ADVANCED

Whoa looks like you're an expert!

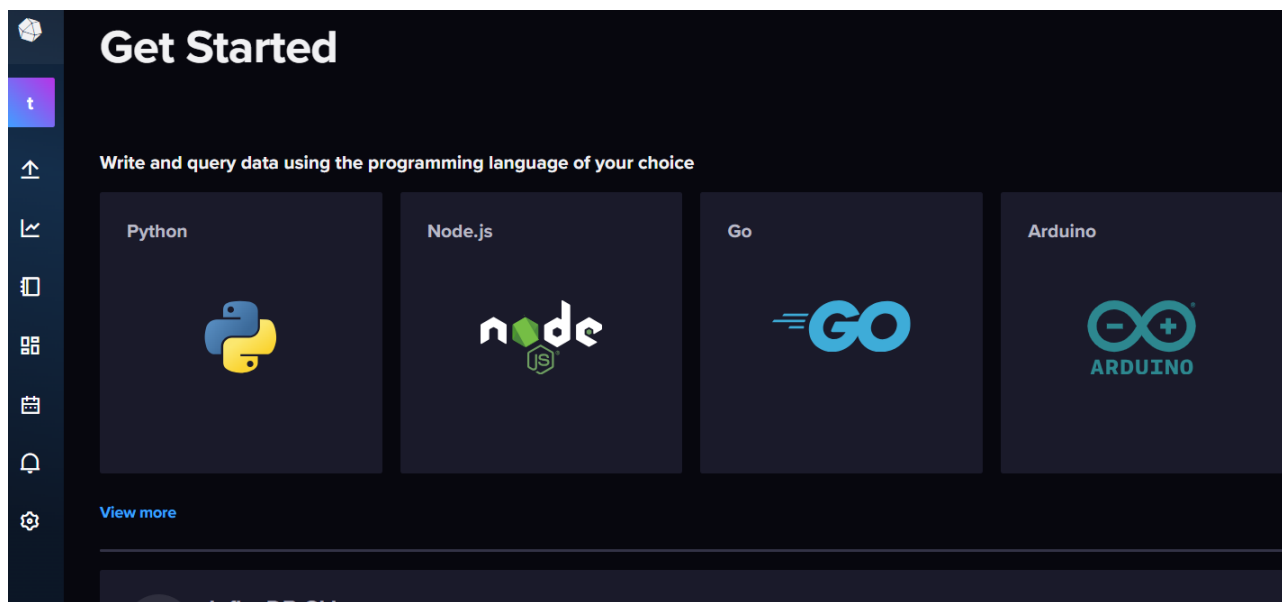
CONFIGURE LATER

I've got this...

保存令牌，此令牌可以启用超级管理员创建用户和组织。

```
1 NKN2TJs1IgiLd_EDouVPwPHNTRCuvbSjg9WhQ6tArR90izCk1tNMYHfkyMv_1Zh5RIpSsKaBR6e4nrT7bauw6Q=
  =
```

点击QUICK START，看到如图所示的页面，说明我们已经开始使用fox这一用户身份和InfluxDB交互了。



3. InfluxDB快速开始

3.1 写入数据

使用InfluxDB行协议写入数据

InfluxDB行协议是InfluxDB数据库独创的一种数据格式，它由纯文本构成，只要数据符合这种格式，就能使用InfluxDB的HTTP API将数据写入数据库。

在InfluxDB行协议中，一条数据和另一条数据之间使用换行符分隔，所以一行就是一条数据。

一行数据由下面4种元素构成：

- **measurement (测量名称)**
 - **必需**
 - 在 InfluxDB 行协议中，测量名称不可省略。
 - 大小写敏感，不可以用下划线_打头
- **Tag Set (标签集)**： 标签应该用在一些值的范围有限（可枚举）的，不太会变动的属性上。比如传感器的类型和 id 等等。在 InfluxDB 中一个 Tag 相当于一个索引。给数据点加上 Tag 有利于将来对数据进行检索。但是如果索引太多了，就会减慢数据的插入速度。
 - **可选**
 - 键值关系使用=表示。多个键值对之间使用英文逗号，分隔
 - 标签的键和值都区分大小写。标签的键不能以下划线_开头
 - 键的数据类型：字符串。值的数据类型：字符串
- **Field Set (字段集)**
 - **必需**

- 一个数据点上所有的字段键值对，键是字段名，值是数据点的值。
- 一个数据点至少要有有一个字段。
- 字段集的键是大小写敏感的。
- 字段键的数据类型：字符串。值的数据类型：浮点数 | 整数 | 无符号整数 | 字符串 | 布尔值

- **Timestamp (时间戳)**

- 可选
- 数据点的 Unix 时间戳，每个数据点都可以制定自己的时间戳。
- 如果时间戳没有指定。那么 InfluxDB 就使用当前系统的时间戳。
- 数据类型：Unix timestamp
- 如果你的数据里的时间戳不是以纳秒为单位的，那么需要在数据写入时指定时间戳的精度。

Syntax:

```
measurementName,tagKey=tagValue fieldKey="fieldValue" 1465839830100400200
```

Measurement	Tag set	Field set	Timestamp

Example:

```
myMeasurement,tag1=value1,tag2=value2 fieldKey="fieldValue" 1556813561098000000
```

行协议中的空格决定了InfluxDB如何解释数据点，第一个未转义的空格将测量值&Tag Set（标签集）与Field Set（字段集）分开。第二个未转义空格将Field Set（字段级）和时间戳分开。

示例：创建Bucket并通过行协议写入数据

在Load Data页面，点击Line Protocol进入InfluxDB行协议格式数据的上传引导页面。

Load Data

SOURCES

BUCKETS

TELEGRAF

SCRAPERS

Q | Search data writing methods...

File Upload

Upload CSV or a line protocol file

Upload a CSV



Line Protocol



具体步骤参考课上演示操作。

使用 Telegraf 写入数据

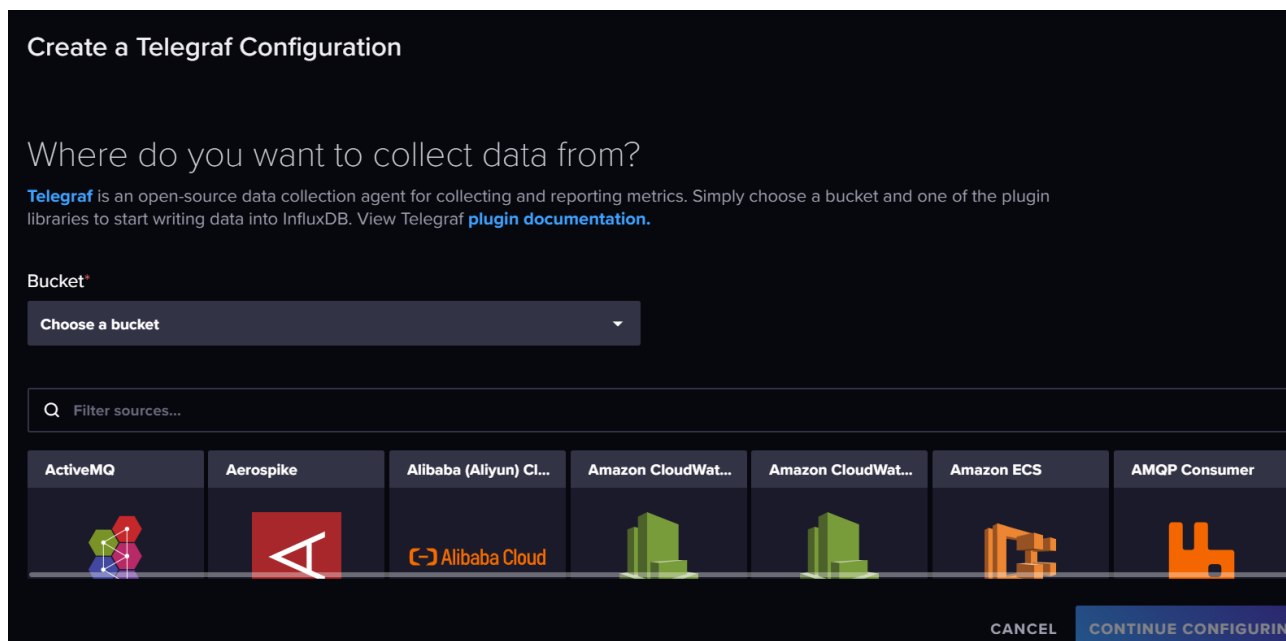
Telegraf是InfluxDB生态中的一个数据采集组件，它可以讲各种时序数据自动采集到InfluxDB。

安装Telegraf

官方文档：<https://docs.influxdata.com/telegraf/v1/install/?t=RedHat+%26amp%3B+CentOS#create-a-configuration-file-with-default-input-and-output-plugins>

```
1 wget https://dl.influxdata.com/telegraf/releases/telegraf-1.30.3_linux_amd64.tar.gz
2 tar xf telegraf-1.30.3_linux_amd64.tar.gz
3 cp ./telegraf-1.30.3/usr/bin/telegraf /usr/local/bin/
```

示例：使用Telegraf将数据收集到InfluxDB



具体步骤参考课上演示操作。

使用 InfluxDB 抓取程序抓取数据

InfluxDB 抓取程序定期从指定目标收集数据，然后将抓取的数据写入 InfluxDB 存储桶。抓取工具可以从任何提供数据的 HTTP (S) 可访问端点收集数据（采用 Prometheus 数据格式）。

Prometheus数据格式的构成

prometheus数据格式主要包含四个元素：

- 指标名称（必需）
- 标签集（可选）：标签集是一组键值对，键是标签的名称，值是具体的标签内容，
- 而且值必须得是字符串。指标名称和标签共同组成索引。
- 值（必须）：必须满足浮点数格式
- 时间戳（可选）：Unix毫秒级时间戳。

```
1 <metric name>{<label name>=<label value>, ...} value [ timestamp ]
```

安装Node Exporter

这里为了能够采集到主机的运行指标如CPU, 内存, 磁盘等信息。我们可以使用Node Exporter。Node Exporter同样采用Golang编写, 并且不存在任何的第三方依赖, 只需要下载, 解压即可运行。可以从<https://prometheus.io/download/>获取最新的node exporter版本的二进制包。

```
1 wget
  https://github.com/prometheus/node_exporter/releases/download/v1.7.0/node_exporter-1.7.0.linux-amd64.tar.gz
2 tar xvzf node_exporter-1.7.0.linux-amd64.tar.gz
3 cd node_exporter-1.7.0.linux-amd64/
4 ./node_exporter
```

访问<http://localhost:9100/metrics>, 可以看到当前node exporter获取到的当前主机的所有监控数据。

```
# HELP go_gc_duration_seconds A summary of the pause duration of garbage collection cycles.
# TYPE go_gc_duration_seconds summary
go_gc_duration_seconds{quantile="0"} 0
go_gc_duration_seconds{quantile="0.25"} 0
go_gc_duration_seconds{quantile="0.5"} 0
go_gc_duration_seconds{quantile="0.75"} 0
go_gc_duration_seconds{quantile="1"} 0
go_gc_duration_seconds_sum 0
go_gc_duration_seconds_count 0
# HELP go_goroutines Number of goroutines that currently exist.
# TYPE go_goroutines gauge
go_goroutines 8
# HELP go_info Information about the Go environment.
# TYPE go_info gauge
go_info{version="go1.21.4"} 1
# HELP go_memstats_alloc_bytes Number of bytes allocated and still in use.
# TYPE go_memstats_alloc_bytes gauge
go_memstats_alloc_bytes 850344
# HELP go_memstats_alloc_bytes_total Total number of bytes allocated, even if freed.
# TYPE go_memstats_alloc_bytes_total counter
go_memstats_alloc_bytes_total 850344
# HELP go_memstats_buck_hash_sys_bytes Number of bytes used by the profiling bucket hash table.
# TYPE go_memstats_buck_hash_sys_bytes gauge
go_memstats_buck_hash_sys_bytes 1.446303e+06
# HELP go_memstats_frees_total Total number of frees.
# TYPE go_memstats_frees_total counter
go_memstats_frees_total 725
# HELP go_memstats_gc_sys_bytes Number of bytes used for garbage collection system metadata.
# TYPE go_memstats_gc_sys_bytes gauge
go_memstats_gc_sys_bytes 2.921712e+06
# HELP go_memstats_heap_alloc_bytes Number of heap bytes allocated and still in use.
# TYPE go_memstats_heap_alloc_bytes gauge
go_memstats_heap_alloc_bytes 850344
# HELP go_memstats_heap_idle_bytes Number of heap bytes waiting to be used.
# TYPE go_memstats_heap_idle_bytes gauge
go_memstats_heap_idle_bytes 1.39264e+06
# HELP go_memstats_heap_inuse_bytes Number of heap bytes that are in use.
# TYPE go_memstats_heap_inuse_bytes gauge
go_memstats_heap_inuse_bytes 2.441216e+06
# HELP go_memstats_heap_objects Number of allocated objects.
# TYPE go_memstats_heap_objects gauge
go_memstats_heap_objects 8017
```

示例：使用抓取程序抓取数据到InfluxDB

Create Scraper

Scrapers collect data from multiple targets at regular intervals and to write to a bucket

Name

Bucket to store scraped metrics

demo2

Target URL

✓

CANCEL

CREATE

具体步骤参考课上演示操作。

3.2 查询数据

Data Explorer

Data Explorer的界面简单分为两个区域，上半部分为数据预览区，下半部分为查询编辑区。

Data Explorer

Graph

CUSTOMIZE

Local

SAVE AS

Query 1

View Raw Data

Past 1m

SCRIPT EDITOR

SUBMIT

FROM

Search buckets

demo2

demo3

demo5

demo_cpu

testdb

Filter

_measurement

Search _measurement tag va

docker

docker_container_blkio

docker_container_cpu

WINDOW PERIOD

CUSTOM

AUTO

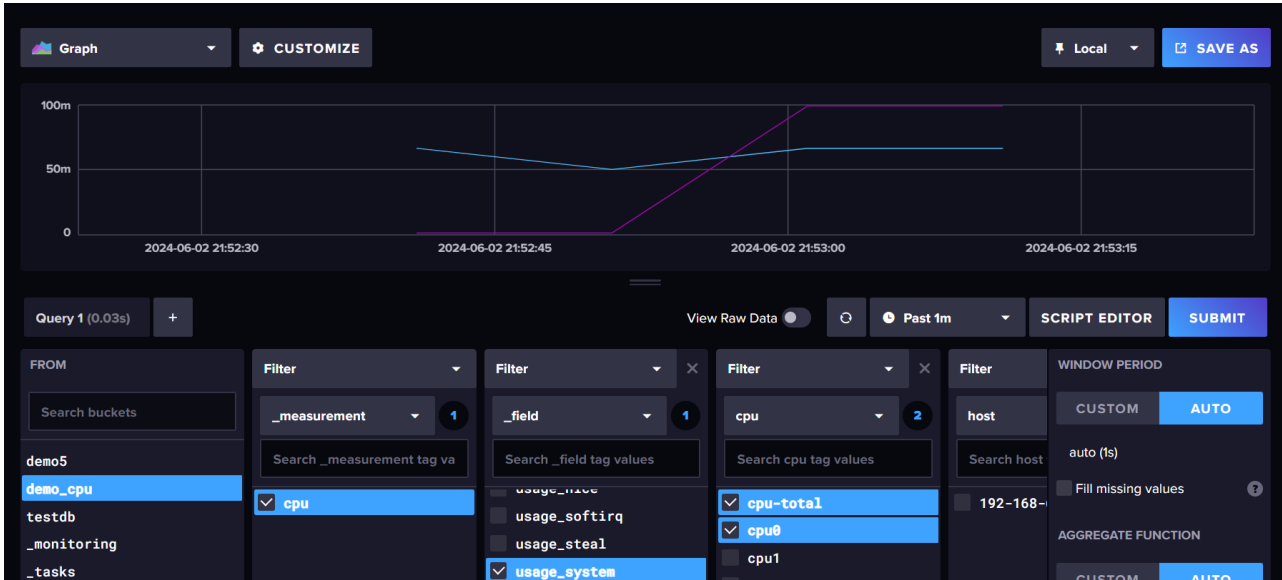
auto (1s)

Fill missing values

AGGREGATE FUNCTION

查询编辑区提供了两种查询工具，一个是查询构造器，一个是FLUX脚本编辑器。进入Data Explorer页面，默认会打开查询构造器。使用查询构造器，你可以通过点按的方式完成查询。它背后的原理其实是根据你的设置，自动生成一条FLUX语句，提交给数据库完成查询。

示例：在Data Explorer使用查询构造器进行查询和可视化

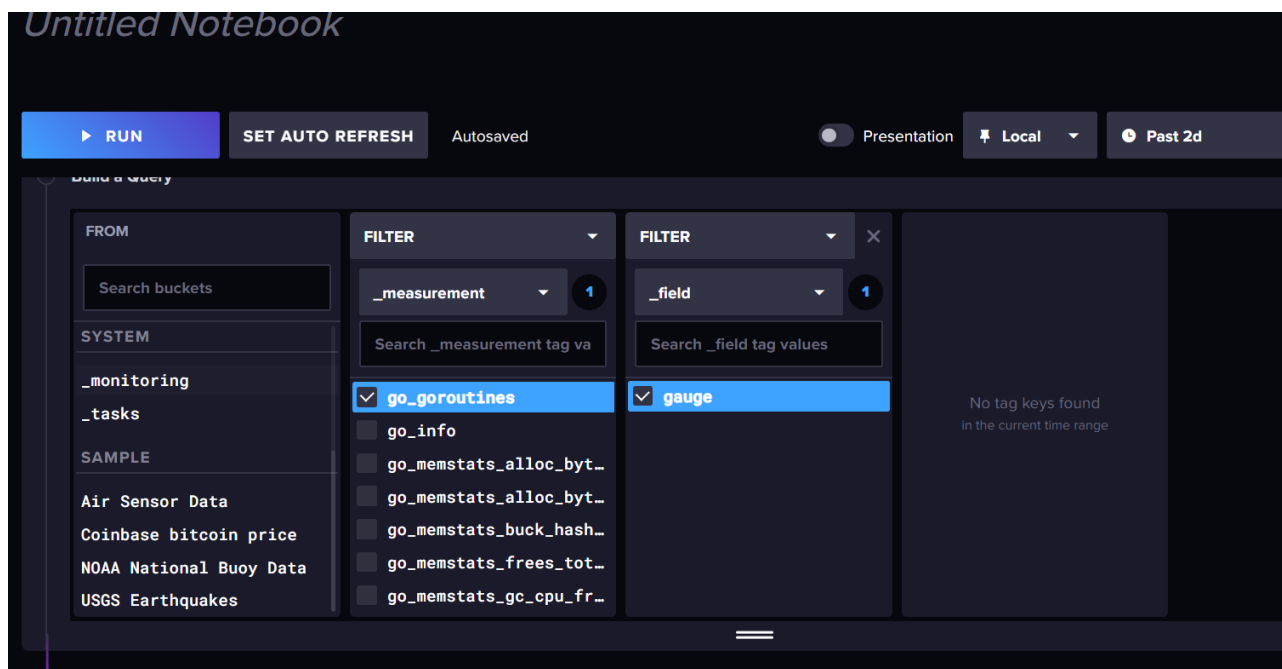


具体步骤参考课上演示操作。

Notebook

Notebook 是 InfluxDB2.x 推出的功能，交互上模仿了 Jupyter NoteBook。它可以用于开发、文档编写、运行代码和展示结果。你可以将 InfluxDB 笔记本视为按照顺序处理数据的集合。每个步骤都由一个“单元格”表示。一个单元格可以执行查询、可视化、处理或将数据写入存储桶等操作。

示例：使用 Notebook 查询和可视化数据



具体步骤参考课上演示操作。

3.3 使用Flux查询InfluxDB

Flux 是一种功能强大的查询语言，特别适用于处理时间序列数据。Flux 具有丰富的操作和函数，使用户能够执行复杂的数据处理、过滤、聚合和计算任务。

Flux入门: <https://docs.influxdata.com/flux/v0/get-started/>

Flux 查询基础知识

使用 Flux 查询 InfluxDB 时，可以使用三个主要函数：

- `from ()`：查询 InfluxDB 存储桶中的数据。
- `range ()`：根据时间限制筛选数据。Flux 需要“有界”查询，即查询 限制在特定的时间范围内。
- `filter ()`：根据列值筛选数据。每一行由 `r` 表示，每一列由 `r` 的一个属性表示。可以应用多个后续筛选器。

`Filter ()` 将每一行读取为名为 `r` 的记录。在 `r` 记录中，每个键值对表示一列及其值。例如：

```
1 r = {
2   _time: 2020-01-01T00:00:00Z,
3   _measurement: "home",
4   room: "Kitchen",
5   _field: "temp",
6   _value: 21.0,
7 }
```

可以使用谓词表达式来计算列的值：

```
1 (r) => r._measurement == "home" // Returns true
2 (r) => r.room == "Kitchen" // Returns true
3 (r) => r._field == "co" // Returns false
4 (r) => r._field == "co" or r._field == "temp" // Returns true
5 (r) => r._value <= 20.0 // Returns false
```

值为true的行包含在filter()输出中。计算结果为false的行将从filter()输出中删除。

- 前向操作符：Flux使用管道前向操作符(|>)将一个函数的输出作为输入管道到下一个函数。

下面的Flux查询返回存储在测量名为home中的co、hum和temp字段，时间戳介于2022-01-01 t8:00:00 z和2022-01-01T20:00:01Z之间。

```
1 from(bucket: "get-started")
2   |> range(start: 2022-01-01T08:00:00Z, stop: 2022-01-01T20:00:01Z)
3   |> filter(fn: (r) => r._measurement == "home")
4   |> filter(fn: (r) => r._field == "co" or r._field == "hum" or r._field == "temp")
```

以下是一些 Flux 查询和计算的基本示例：

查询数据

1. 选择数据点：

```
1 from(bucket: "your_bucket")
2   |> range(start: -1h)
3   |> filter(fn: (r) => r._measurement == "your_measurement")
```

2. 选择特定字段：

```
1 from(bucket: "your_bucket")
2   |> range(start: -1h)
3   |> filter(fn: (r) => r._measurement == "your_measurement")
4   |> filter(fn: (r) => r._field == "your_field")
```

数据处理和计算

1. 计算平均值:

```
1 from(bucket: "your_bucket")
2   |> range(start: -1h)
3   |> filter(fn: (r) => r._measurement == "your_measurement")
4   |> filter(fn: (r) => r._field == "your_field")
5   |> mean()
```

2. 计算总和:

```
1 from(bucket: "your_bucket")
2   |> range(start: -1h)
3   |> filter(fn: (r) => r._measurement == "your_measurement")
4   |> filter(fn: (r) => r._field == "your_field")
5   |> sum()
```

3. 计算差异:

```
1 from(bucket: "your_bucket")
2   |> range(start: -1h)
3   |> filter(fn: (r) => r._measurement == "your_measurement")
4   |> difference()
```

数据过滤和筛选

1. 按标签过滤:

```
1 from(bucket: "your_bucket")
2   |> range(start: -1h)
3   |> filter(fn: (r) => r._measurement == "your_measurement" and r.your_tag ==
   "desired_value")
```

2. 按条件过滤:

```
1 from(bucket: "your_bucket")
2   |> range(start: -1h)
3   |> filter(fn: (r) => r._measurement == "your_measurement" and r._value > 10.0)
```

3. 按时间范围过滤:

```
1 from(bucket: "your_bucket")
2   |> range(start: -1h, stop: now())
3   |> filter(fn: (r) => r._measurement == "your_measurement")
```

3.4 使用influx命令行工具

官方文档: <https://docs.influxdata.com/influxdb/v2/reference/cli/influx/>

github地址: <https://github.com/influxdata/influx-cli>

从InfluxDB 2.1版本之后, influx命令行和InfluxDB数据库服务程序是分开打包的, 所以安装InfluxDB并不会附带influx命令行工具。用户必须单独安装influx命令行工具。influx命令行工具包含很多管理influxDB的命令。包括存储桶、组织、用户、任务等。

安装influx-cli

```
1 wget https://download.influxdata.com/influxdb/releases/influxdb2-client-2.7.5-linux-
  amd64.tar.gz
2 tar xvzf ./influxdb2-client-2.7.5-linux-amd64.tar.gz
3 sudo cp ./influx /usr/local/bin/
```

```
[root@192-168-65-223 ~]# tar xvzf ./influxdb2-client-2.7.5-linux-amd64.tar.gz
./
./LICENSE
./README.md
./influx
[root@192-168-65-223 ~]# sudo cp ./influx /usr/local/bin/
[root@192-168-65-223 ~]# influx
NAME:
    influx - Influx Client

USAGE:
    influx [command]

HINT: If you are looking for the InfluxQL shell from 1.x, run "influx v1 shell"

COMMANDS:
    version          Print the influx CLI version
    write            Write points to InfluxDB
    bucket           Bucket management commands
    completion       Generates completion scripts
    query            Execute a Flux query
    config           Config management commands
    org, organization Organization management commands
    delete           Delete points from InfluxDB
    user            User management commands
```

配置influx-cli

influx命令行工具是你每执行一次操作时，调用一次命令。并不是开启一个持续性的会话。而influx其实底层还是封装的对InfluxDB的服务进程的http请求。也就是它还是需要配置Token什么的来获取授权。所以，为了避免以后每次请求的时候都在命令行里面写一遍token。我们应该先去搞个配置文件。使用下面的命令可以创influx命令行的配置：

```
1 #使用influx config create命令创建 CLI 配置并将其设置为活动状态
2 influx config create --config-name influx_conf \
3   --host-url http://192.168.65.223:8086 \
4   --org tuling \
5   --token
   NKN2TJs1IgiLd_EDouVPwPHNTRCuvbSjg9WhQ6tArR90izCkltNMYHfkyMv_lZh5RIpSsKaBR6e4nrT7bauw6Q=
   = \
6   --active
```

这个命令其实会在~/influxdbv2/目录下创建一个configs文件，这个文件中，就是我们命令行中写的各项配置。如图所示：

```
[root@192-168-65-223 ~]# cat .influxdbv2/configs
[influx_conf]
url = "http://192.168.65.223:8086"
token = "NKN2TJs1IgiLd_EDouVPwPHNTRCuvbSjg9WhQ6tArR90izCkltNMYHfkyMv_LZh5RIpSsKaBR6e4nrT7bauw6Q=="
org = "tuling"
active = true
#
# [eu-central]
# url = "https://eu-central-1-1.aws.cloud2.influxdata.com"
# token = "XXX"
# org = ""
```

如果后续需要更改配置，把influx config create换成influx config update即可。

使用influx-cli

参考文档: <https://docs.influxdata.com/influxdb/v2/reference/cli/influx/>

```
[root@192-168-65-223 ~]# influx bucket list
ID Name Retention Shard group duration Organization ID Schema Type
e2c184254c41b662 _monitoring 168h0m0s 24h0m0s 1e5bf08b064b81f6 implicit
227085e642524766 _tasks 72h0m0s 24h0m0s 1e5bf08b064b81f6 implicit
aa042856aecfed94 demo2 infinite 168h0m0s 1e5bf08b064b81f6 implicit
4ccc6baaa8879c82 demo3 infinite 168h0m0s 1e5bf08b064b81f6 implicit
4ec5bdd7dff4f770 demo5 168h0m0s 24h0m0s 1e5bf08b064b81f6 implicit
cd513bac74ee7a65 demo_cpu 720h0m0s 24h0m0s 1e5bf08b064b81f6 implicit
a16210cdc2edf847 testdb infinite 168h0m0s 1e5bf08b064b81f6 implicit
[root@192-168-65-223 ~]# influx config list
Active Name URL Org
* influx_conf http://192.168.65.223:8086 tuling
[root@192-168-65-223 ~]# influx config list --json
{
  "influx_conf": {
    "url": "http://192.168.65.223:8086",
    "token": "NKN2TJs1IgiLd_EDouVPwPHNTRCuvbSjg9WhQ6tArR90izCkltNMYHfkyMv_LZh5RIpSsKaBR6e4nrT7bauw6Q==",
    "org": "tuling",
    "active": true
  }
}
```

兼容v1，使用 InfluxQL查询数据

InfluxQL 是一种类似于 SQL 的查询语言，类似于大多数 SQL 语言，但 专门设计用于从 InfluxDB 0.x 和 1.x 查询时间序列数据。InfluxDB UI 不支持 InfluxQL。

```
1 influx v1 shell
2 SELECT co,hum,temp,room FROM "demo2".autogen.home WHERE time >= '2024-06-03T11:53:31.000Z' AND time <= '2024-06-03T11:55:31.000Z'
```

Interactive Table View (press q to exit mode, shift+up/down to navigate tables):

Name: home

index	time	co	hum	temp	room
1	1717415611000000000.000000000	0.000000000	35.900000000	21.000000000	Kitchen
2	1717415643000000000.000000000	0.000000000	36.200000000	23.000000000	Kitchen

6 Columns, 2 Rows, Page 1/1
Table 1/1, Statement 1/1

3.5 JAVA操作InfluxDB

参考资料: <https://github.com/influxdata/influxdb-client-java>

引入依赖

```
1 <dependency>
2   <groupId>com.influxdb</groupId>
3   <artifactId>influxdb-client-java</artifactId>
4   <version>6.6.0</version>
5 </dependency>
```

示例代码:


```
1 public class InfluxDB2Example {
2     public static void main(final String[] args) {
3         String token =
4             "NKN2TJs1IgiLd_EDouVPwPHNTRCuvbSjg9WhQ6tArR90izCkltNMYHfkyMv_lZh5RIpSsKaBR6e4nrT7bauw6Q
5             ==";
6
7         String bucket = "demo2";
8         String org = "tuling";
9
10        InfluxDBClient client =
11            InfluxDBClientFactory.create("http://192.168.65.223:8086", token.toCharArray());
12
13        //基于行协议写数据
14
15        String data = "mem,host=host1 used_percent=23.43234543";
16        WriteApiBlocking writeApi = client.getWriteApiBlocking();
17        writeApi.writeRecord(bucket, org, WritePrecision.NS, data);
18
19        //基于point写数据
20
21        Point point = Point
22            .measurement("mem")
23            .addTag("host", "host1")
24            .addField("used_percent", 23.43234543)
25            .time(Instant.now(), WritePrecision.NS);
26
27        WriteApiBlocking writeApi = client.getWriteApiBlocking();
28        writeApi.writePoint(bucket, org, point);
29
30        //基于pojo写数据
31
32        Mem mem = new Mem();
33        mem.host = "host1";
34        mem.used_percent = 23.43234543;
35        mem.time = Instant.now();
36
37        WriteApiBlocking writeApi = client.getWriteApiBlocking();
38        writeApi.writeMeasurement(bucket, org, WritePrecision.NS, mem);
39    }
40 }
```

```

36      // 执行Flux查询
37      String query = "from(bucket: \"demo2\")\n" +
38          "    |> range(start: -1h )\n" +
39          "    |> filter(fn: (r) => r[\"_measurement\"] == \"mem\")";
40      List<FluxTable> tables = client.getQueryApi().query(query, org);
41
42      for (FluxTable table : tables) {
43          for (FluxRecord record : table.getRecords()) {
44              System.out.println(record.getValues());
45          }
46      }
47
48      client.close();
49
50
51  }
52
53
54  }

```

3.6 使用 InfluxDB 模板

InfluxDB 模板是一份 yaml 风格的配置文件。它包含了一套完整的仪表盘、Telegraf 配置和报警配置。InfluxDB 模板力图保证开箱即用，把 yaml 文件下载下来，往 InfluxDB 里一导，从数据采集一直到数据监控报警就全部为你创建好。

InfluxDB 官方在 github 上收录了一批模板。开发前可以在这里逛一逛，看有没有可以直接拿来用的。

<https://github.com/influxdata/community-templates>

示例：使用模板快速部署

在这节示例中，我们会使用社区模板快速创建一套 docker 的监控模板。要完成这个示例，你需要提前掌握 docker 的相关知识。

使用InfluxDB UI安装docker模版

Settings

VARIABLESTEMPLATESLABELSSECRETS

1Find a template then return here to install it

BROWSE COMMUNITY TEMPLATES

2Paste the URL of the Template's resource manifest file

Every template has a file that controls what gets installed, which is usually a YAML or JSON file

https://raw.githubusercontent.com/influxdata/community-templates/master/docker/docker.yml

LOOKUP TEMPLATE

You're good to go!

安装成功后，显示

Installed Templates

TEMPLATE NAME	INSTALLED RESOURCES	INSTALL DATE	LINKS
docker	▼ 20 Resources Telegraf lucid-goodall-d4d017 Dashboard earning-boyd-94d001 NotificationRule gifted-haslett-14d008 NotificationEndpoint earnest-ishizaka-14d006 Check crumbling-meitner-14d002 Check ecstatic-mendeleev-d4d01b Check frosty-greider-14d004 Check gracious-bose-14d000 Bucket docker Label adoring-pasteur-d4d00d Label agreeing-goldberg-d4d013 Label dreamy-heyrovsky-d4d011 Label elastic-morse-d4d001 Label elegant-yonath-d4d015 Label gallant-sanderson-d4d003 Label goofy-yalow-d4d005		

 Mon Jun 03 2024 | VIEW README |

查看安装结果

现在，我们打开 InfluxDB 的 Web UI，看一下我们模板的导入效果。

- 创建的存储桶，名为 docker。
- 一个名为 Docker Monitor 的 Telegraf 配置文件，这个配置文件可能需要根据你 Docker 的配置进行一些修改
- 模板还帮我们创建了一个名为 Docker 仪表盘，只不过我们现在的 Bucket 里面还没有数据
- 模板还帮我们设置了 4 个报警规则，根据题目的描述，分别是
 - 容器 CPU 使用率持续 15 分钟超过 80%
 - 容器硬盘使用率超过 80%
 - 容器的内存使用率持续 15 分钟超过 80%
 - 容器没有以 0 状态（正常结束）退出。

运行 Telegraf 采集数据

Docker Monitoring
A set of plugins for monitoring your system.

Setup Instructions

+ inputs.docker X inputs.mysql X
inputs.net X inputs.kernel X

docker_conf
监控docker
Setup Instructions

terminal window to set an environment variable with your API token.

```
export INFLUX_TOKEN=MX58AV-vTJpm5T6LJG_iR1SoI6K1Syd2S3C1z--ZsrmquLF91XZf004CYaQ4b1s6SMfhcVEwR-2PIuabnsioXA==
```

COPY TO CLIPBOARD GENERATE NEW API TOKEN CLI

3. Start Telegraf

Finally, you can run the following command to start the Telegraf agent running on your machine.

```
telegraf --config http://192.168.65.223:8086/api/v2/telegrafs/0d238e6152884000
```

COPY TO CLIPBOARD CLI

按照 docker 模板的要求，在运行 telegraf 之前，我们需要声明 INFLUX_TOKEN、INFLUX_HOST 和 INFLUX_ORG 三个变量。

Docker Monitoring

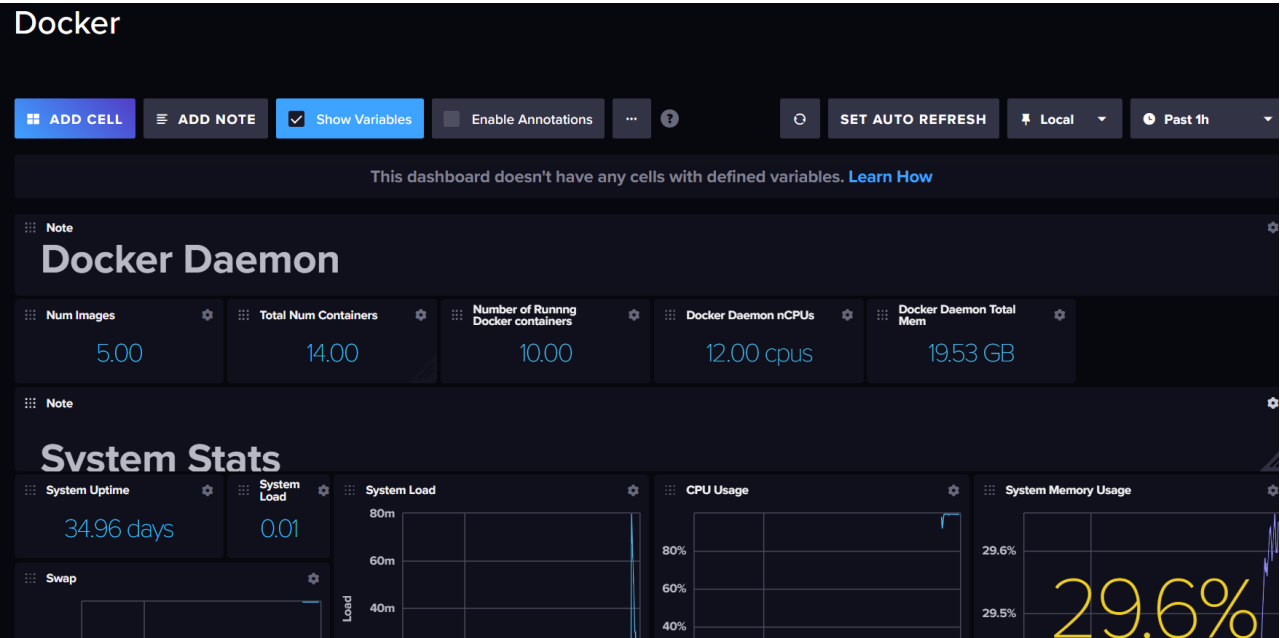
```
24 #####
25 #                                     OUTPUT PLUGINS
26 #####
27
28 # Configuration for sending metrics to InfluxDB
29 [[outputs.influxdb_v2]]
30     urls = ["$INFLUX_HOST"]
31     token = "$INFLUX_TOKEN"
32     organization = "$INFLUX_ORG" ## Destination bucket to write into.
33     bucket = "docker"
34
```

```
1 export INFLUX_TOKEN=MX58AV-vTJpm5T6LJG_iR1SoI6K1Syd2S3C1z--ZsrmquLF91XZf004CYaQ4b1s6SMfhcVEwR-2PIuabnsioXA==
2 export INFLUX_HOST=http://localhost:8086/
3 export INFLUX_ORG=tuling
4 telegraf --config http://192.168.65.223:8086/api/v2/telegrafs/0d238e6152884000
```

如下图所示，telegraf 成功启动。

```
[root@192-168-65-223 ~]# export INFLUX_TOKEN=MX58AV-vTJpm5T6LJG_iR1SoI6KlSyd2S3C1z--ZsrmqULF91XZf004CYaQ4bIs6SMfhcVEwR-2PIuabnsioXA==
[root@192-168-65-223 ~]# export INFLUX_HOST=http://localhost:8086/
[root@192-168-65-223 ~]# export INFLUX_ORG=tuling
[root@192-168-65-223 ~]# telegraf --config http://192.168.65.223:8086/api/v2/telegrafs/0d238e6152884000
2024-06-03T13:39:59Z I! Loading config: http://192.168.65.223:8086/api/v2/telegrafs/0d238e6152884000
2024-06-03T13:39:59Z W! DeprecationWarning: Option "container_names" of plugin "inputs.docker" deprecated since version 1.4.0 and will be removed in 2
.0.0: use 'container_name_include' instead
2024-06-03T13:39:59Z W! DeprecationWarning: Option "perdevice" of plugin "inputs.docker" deprecated since version 1.18.0 and will be removed in 2.0.0:
use 'perdevice_include' instead
2024-06-03T13:39:59Z W! DeprecationWarning: Option "total" of plugin "inputs.docker" deprecated since version 1.18.0 and will be removed in 2.0.0: use
'total_include' instead
2024-06-03T13:39:59Z I! Starting Telegraf 1.30.3 brought to you by InfluxData the makers of InfluxDB
2024-06-03T13:39:59Z I! Available plugins: 233 inputs, 9 aggregators, 31 processors, 24 parsers, 60 outputs, 6 secret-stores
2024-06-03T13:39:59Z I! Loaded inputs: cpu disk diskio docker kernel mem net processes swap system
2024-06-03T13:39:59Z I! Loaded aggregators:
2024-06-03T13:39:59Z I! Loaded processors:
2024-06-03T13:39:59Z I! Loaded secretstores:
2024-06-03T13:39:59Z I! Loaded outputs: influxdb_v2
2024-06-03T13:39:59Z I! Tags enabled: host=192-168-65-223
2024-06-03T13:39:59Z W! DeprecationWarning: Value "false" for option "ignore_protocol_stats" of plugin "inputs.net" deprecated since version 1.27.3 an
d will be removed in 1.36.0: use the 'inputs.nstat' plugin instead for protocol stats
```

查看仪表盘



3.7 使用InfluxDB执行定时任务

InfluxDB 任务是一个定时执行的 FLUX 脚本，它先查询数据，然后以某种方式进行修改或聚合，然后将数据写回 InfluxDB 或执行其他操作。

示例：定时调用其他应用的接口

1) 准备一个测试应用



simpleHttpPostS... .zip
1.63KB

```
1 # 解压后进入对应目录
2 go build
3 simpleHttpPostServer -p 8000
```

2) 创建任务

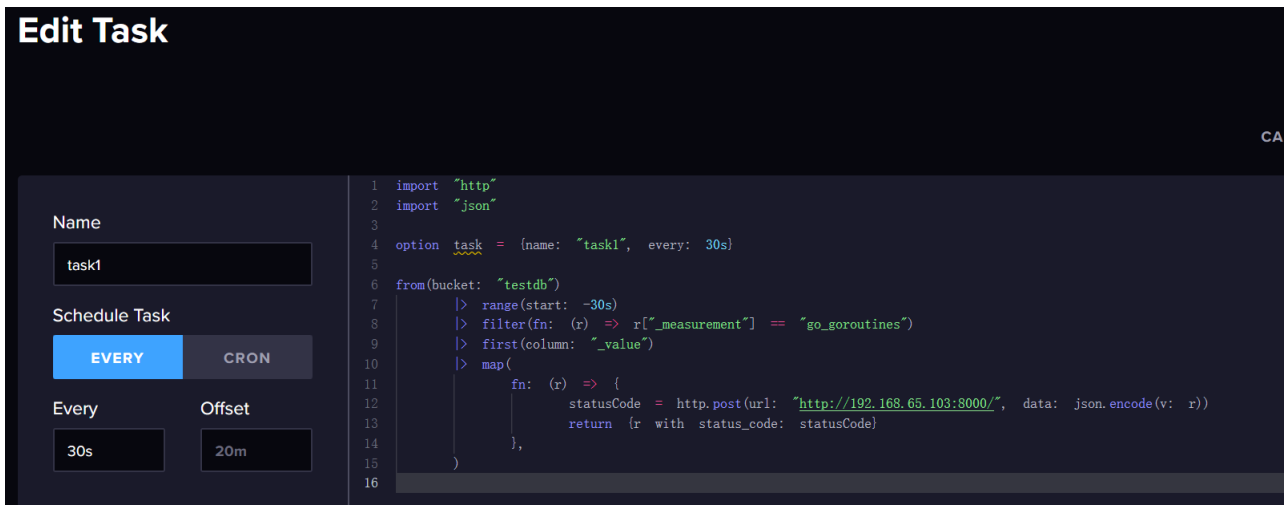
flux提交请求参考文档: <https://docs.influxdata.com/flux/v0/stdlib/http/post/>

我们的定时任务要实现下面的需求。

- (1) 每 30s 调度一次
- (2) 调用应用的接口

```
1 import "http"
2 import "json"
3
4 option task = {name: "task1", every: 30s}
5
6 from(bucket: "testdb")
7   |> range(start: -30s)
8   |> filter(fn: (r) => r["_measurement"] == "go_goroutines")
9   |> first(column: "_value")
10  |> map(
11    fn: (r) => {
12      statusCode = http.post(url: "http://192.168.65.103:8000/", data:
13        json.encode(v: r))
14      return {r with status_code: statusCode}
15    },
16  )
```

3) 在webUI界面配置定时任务



```
D:\迅雷下载\simpleHttpPostServer-0.0.1>simpleHttpPostServer.exe -p 8000
{"_field": "gauge", "_measurement": "go_goroutines", "_start": "2024-06-04T06:15:55Z", "_stop": "2024-06-04T06:16:25Z", "_time": "2024-06-04T06:15:59.4048438Z", "_value": 1317}
{"_field": "gauge", "_measurement": "go_goroutines", "_start": "2024-06-04T06:16:00Z", "_stop": "2024-06-04T06:16:30Z", "_time": "2024-06-04T06:16:09.405679274Z", "_value": 1319}
{"_field": "gauge", "_measurement": "go_goroutines", "_start": "2024-06-04T06:16:30Z", "_stop": "2024-06-04T06:17:00Z", "_time": "2024-06-04T06:16:39.405440735Z", "_value": 1319}
```

3.8 使用 InfluxDB 搭建报警系统

在 Web UI 的左侧工具栏中，点击 Alerts 按钮，会打开一个报警的配置页面。上方的选项 栏 中 显示着 CHECKS（检查）、NOTIFICATION ENDPOINTS（报警终端）和 NOTIFICATION RULES（报警规则）分别对应着 InfluxDB 进行报警所需要的 3 个组件。

三个组件的功能分别如下：

- CHECKS（检查）：它其实也是一种定时任务，我们可以称之为检查任务。检查任务会从目标存储桶中读取部分数据然后进行阈值检查，并最终出 4 类信号。CRIT（严重）、WARN（警戒）、INFO（信息）和 OK（良好）
- NOTIFICATION ENDPOINTS（报警终端）：是一个向指定地址发送报警信号的组件。
- NOTIFICATION RULES（报警规则）：它可以指定哪些 Check 出问题了发送微信报警，哪些 Check 出问题了可以发邮件通知。它相当于 Check 与报警终端之间的路由。

示例：模拟对一氧化碳浓度的报警

假设我们现在有一个可以采集一氧化碳浓度的传感器，这个传感器通过物联网网络每隔一段时间就向我们部署在服务器上的 InfluxDB 插入一条数据，格式如下：

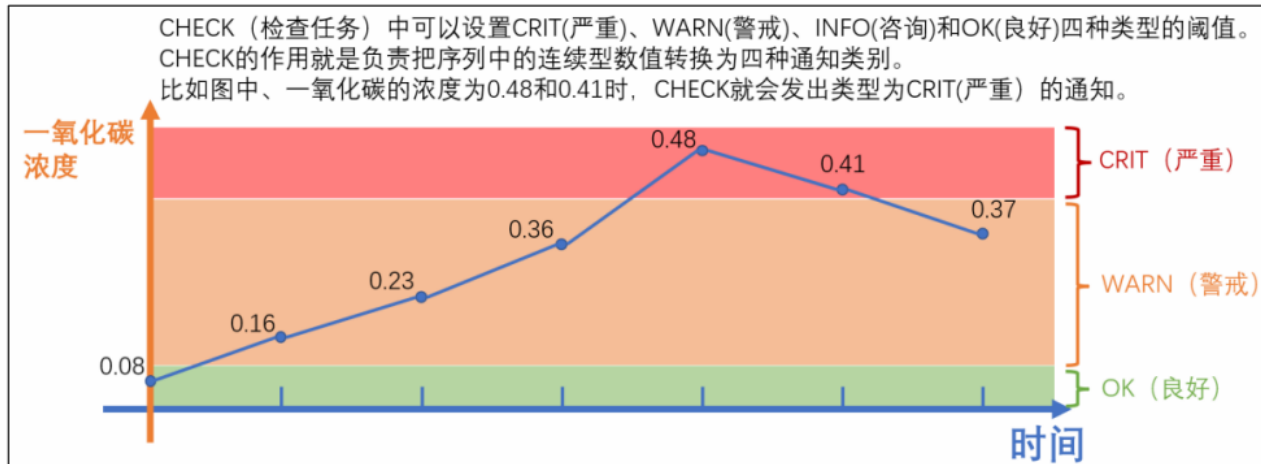
```
1 co,num=01 value=0.001 1664851126000
```

现在，我们希望使用 InfluxDB 能够完成下述的报警功能。

- 当 CO 浓度大于 0.04 的时候发出 CRIT（严重）级别的通知信号。
- 当 CO 浓度介于 0.04 和 0.01 之间的时候发出 WARN（警戒）级别的通知信号。

- 当 CO 浓度低于 0.01 的时候发出 OK（良好）级别的通知信号。

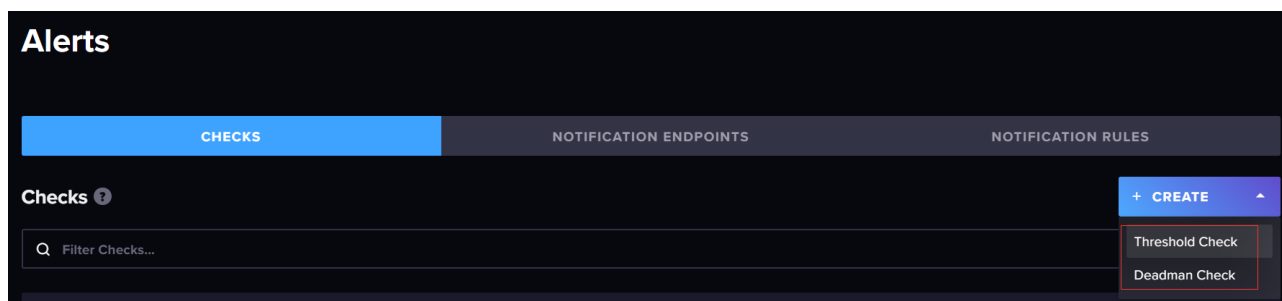
最终，当 CO 浓度超标时，我们希望相关的工作人员能够收到一通电话，以便对事故做出快速响应。



1) 可以利用行协议写入测试数据

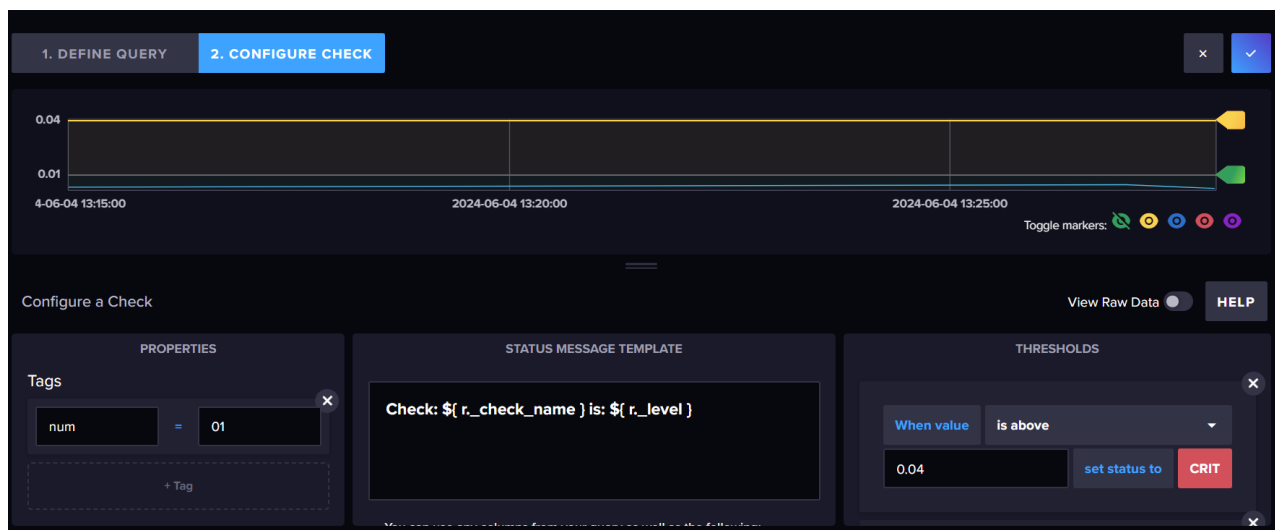
```
1 co,num=01 value=0.005
2 co,num=01 value=0.015
3 co,num=01 value=0.035
4 co,num=01 value=0.045
```

2) 创建检查 (CHECK)



- THRESHOLD CHECK（阈值检查）：这类检查任务主要是去判断数据有没有超出某种阈值限定。
- Deadman Check（死人检查）：这类检查任务是去判断某个序列下多长时间没有写入新的数据了。你也可以设定一个值，比如一旦超过 30s 某个序列还没有数据入库，就发出一个警戒信号。

此处，我们选择 Threshold Check，创建一个阈值检查。



3) 配置报警终端

Edit a Notification Endpoint

Destination

HTTP

Name

check_endpoint

Description

co报警

HTTP Options

HTTP Method

POST

Auth Method

none

URL

http://192.168.65.103:8000/

4) 配置报警规则

Edit this Notification Rule

About

Name

notifyRule

Schedule Every

5s

Offset

0s

Conditions

When status

is equal to

● CRIT

+ Tag Filter

Message

Notification Endpoint

check_endpoint

具体步骤参考课上演示操作。